

Introducción

Diseño de Sistemas Operativos

Grado en Ing. de Computadores, 3er Curso

Depto. de Arquitectura de Computadores
Universidad de Málaga

Contenidos

- Función del SO
- Servicios del SO
- Diseño del SO
  - Kernel
  - Llamadas al Sistema
  - Tolerancia a fallos
  - Protección
  - Virtualización
  - Mecanismos y políticas
  - Kernels monolíticos / Microkernels
  - Módulos cargables (Linux)
  - Otros tipos de kernels

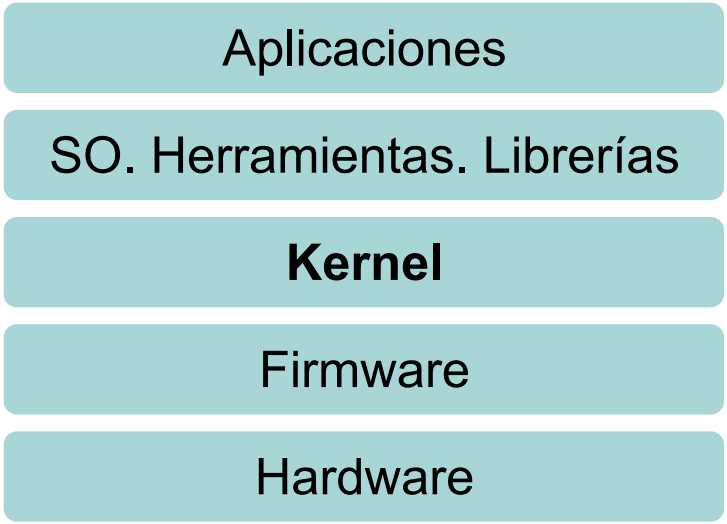
Función del SO

- Interfaz entre las aplicaciones y el HW
  - Llamadas al sistema (SCI)
- Administración de recursos
- Interfaz con el usuario (UI)
  - Línea de comandos – CLI
  - Gráfico – GUI

Servicios que proporciona

- El **kernel** a través del SCI (llamadas al sistema) permite:
  - Controlar la ejecución de procesos
    - Creación, terminación, suspensión, comunicación, …
  - Planificar los procesos
  - Gestionar memoria para los procesos
  - Gestionar espacio en almacenamiento secundario – sistema de ficheros
  - Acceder a dispositivos
    - Terminales, discos, cintas, disp. red, usb, …

Diseño del S.O.

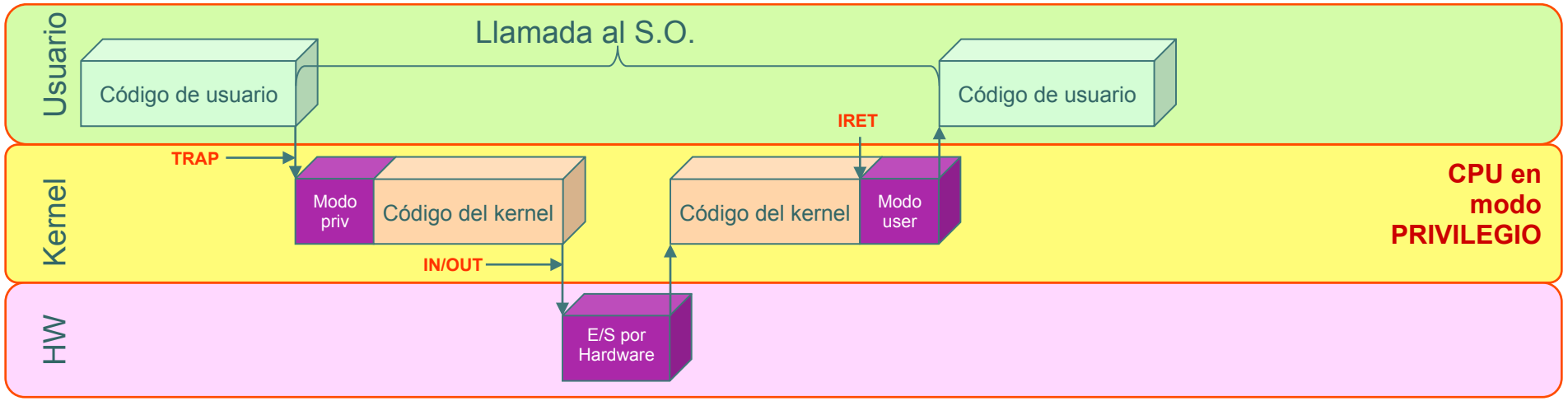


- La mayoría de SS.OO. depende del concepto de **kernel**
  - Consecuencia de un diseño basado en capas de abstracción
  - Parte de obligatoria existencia, el de más bajo nivel del software

Kernel

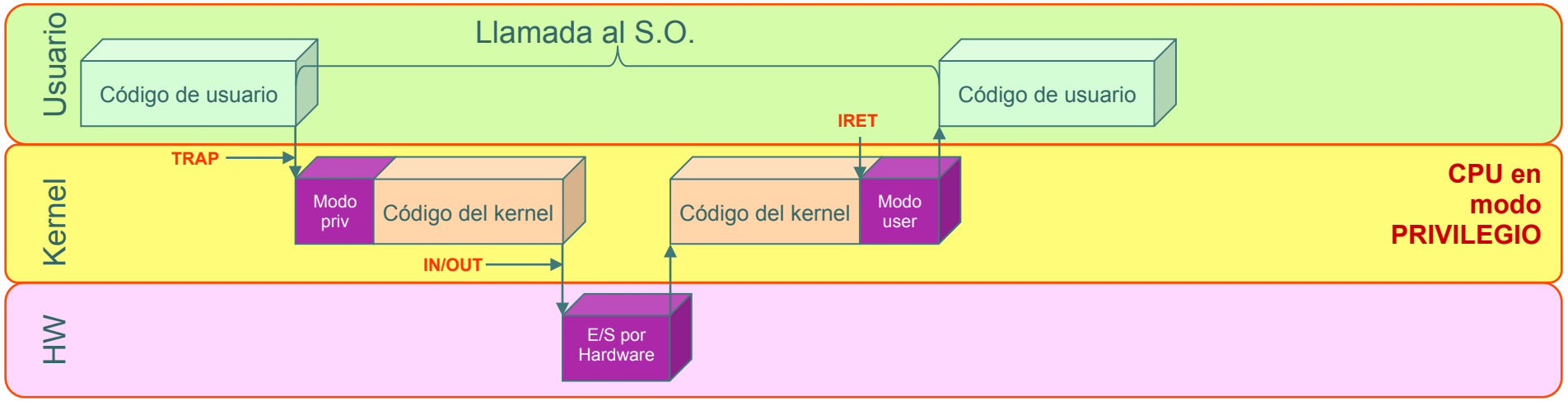
- Parte crítica respecto al rendimiento
  - Rutinas de ejecución muy frecuente
- Complejidad
  - No puede utilizar mecanismos de alto nivel que implementa para los procesos (p.e., mem. Virtual)
  - El código del Kernel es reentrante
- Funcionamiento
  - Una vez cargado por el boot loader e inicializado, el kernel lanza el primer proceso de usuario (init)
  - ya no vuelve a ejecutarse más como «programa»
  - Sólo responde a peticiones de procesos (SCI) o interrupciones del HW

Llamadas al sistema (SCI)



- Acceso a los servicios del kernel
- Interface usado por los procesos
- Librería en C (API) que implementa las llamadas a las system calls
- El programador puede que no las use directamente: librerías del lenguaje de alto nivel

Llamadas al sistema (SCI)



- Mecanismo de invocación dependiente de la arquitectura. Para x86:
  - No se pueden invocar normalmente (`Call`) por la protección de memoria
  - Trap / interrupción software (`int 80h, int 2eh / iret`)
  - Instrucción especial (cambiar SS:ESP / CS:EIP / anillo 0)
    - **`SYSENTER/SYSEXIT`** (Intel a partir de Pentium II)
    - **`SYSCALL/SYSRET`** (AMD a partir de AMD K7)

Tolerancia a fallos

- Minimizar TCB (Trusted Computing Base)
  - Conjunto de hard, firm y soft-ware críticos para la seguridad
  - Bug dentro del TCB compromete la seguridad del sistema completo
- Principio de mínimo privilegio
  - Cada componente debe ser capaz de acceder sólo y exclusivamente a la información y recursos necesarios para su propósito legítimo
- Separación de privilegios
  - Asumir privilegios en el último momento posible y descartarlos en cuanto no sean necesarios
- Proteger acceso a los privilegios

Protección

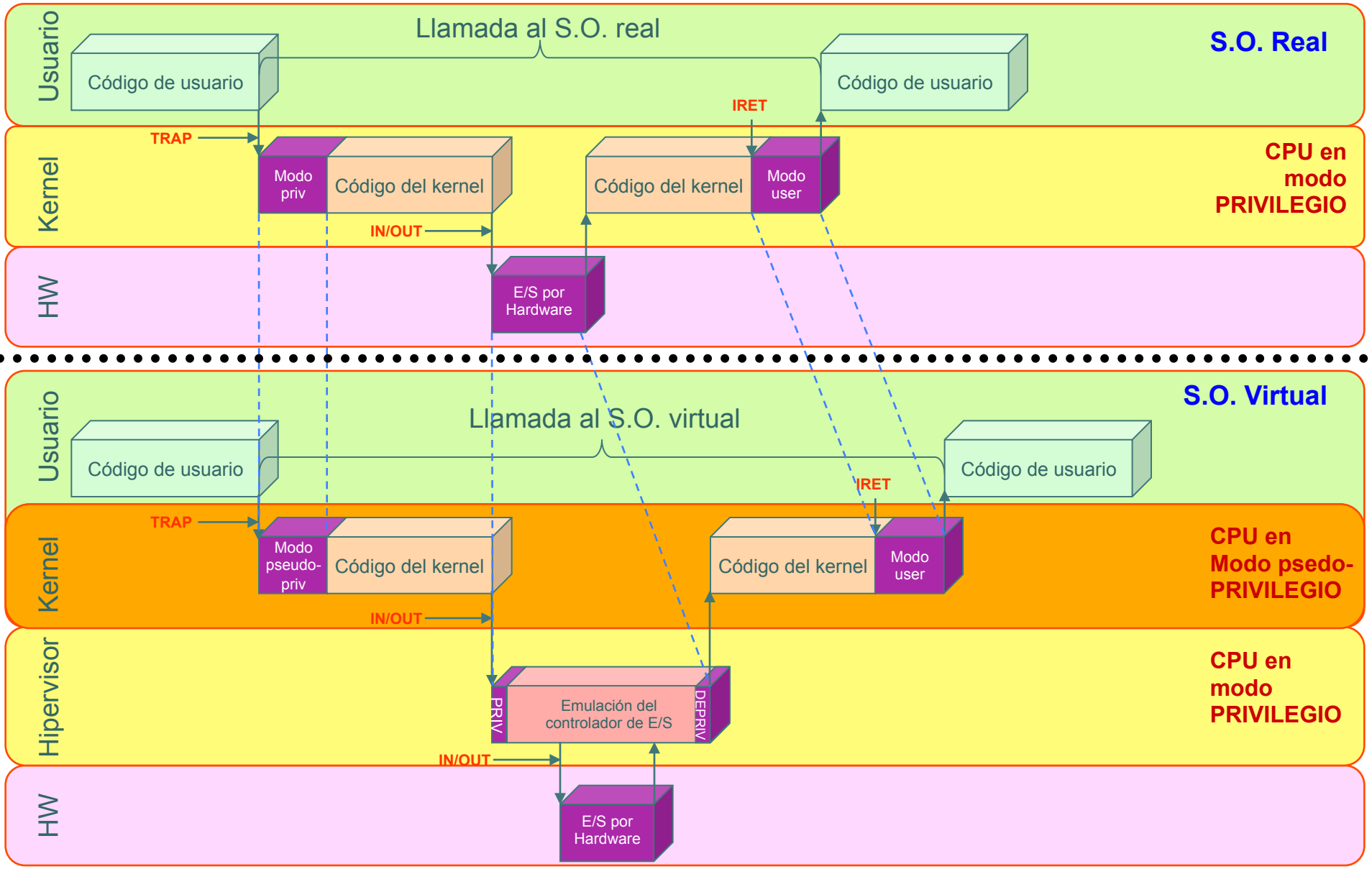


- Protección basada en anillos
  - Dividir en niveles de privilegio el acceso al HW
- Anillo 0
  - Más fiable, más privilegiado
- Anillo 3
  - Menos fiable, menos privilegiado
- Soportado por la arquitectura del procesador: modos de ejecución para la CPU

Protección

- Windows y Linux solo explotan dos niveles de ejecución
  - Kernel (anillo 0)
  - Usuario (anillo 3)
- Cambiar de nivel (llamada al sistema) es caro. **`getpid()`** = 1000~1500 ciclos
- Últimamente aparece el anillo -1, para soportar virtualización (Intel VT, AMD-V)
  - Hipervisores, virtualización hardware

Virtualización



Mecanismos y políticas

- Principio de separación del mecanismo y la política (microkernels)
  - Principio de diseño en informática
  - Los **mecanismos** (parte de la implementación que controla la ejecución de operaciones y asignación de recursos) no deben dictar o limitar las **políticas** con las que se toman las decisiones sobre qué operaciones ejecutar y cómo asignar los recursos
- Ejemplo: mecanismos de planificación y políticas de planificación de procesos

Microkernels

- Aplica el principio de separación del mecanismo y la política
  - El kernel implementa solo algunas políticas básicas y sus mecanismos
  - Permite que otras políticas y servicios se implementen por servidores en el espacio del usuario
- Más fácil de mantener, flexibilidad para personalizar políticas
- Menos eficiente (más capas)

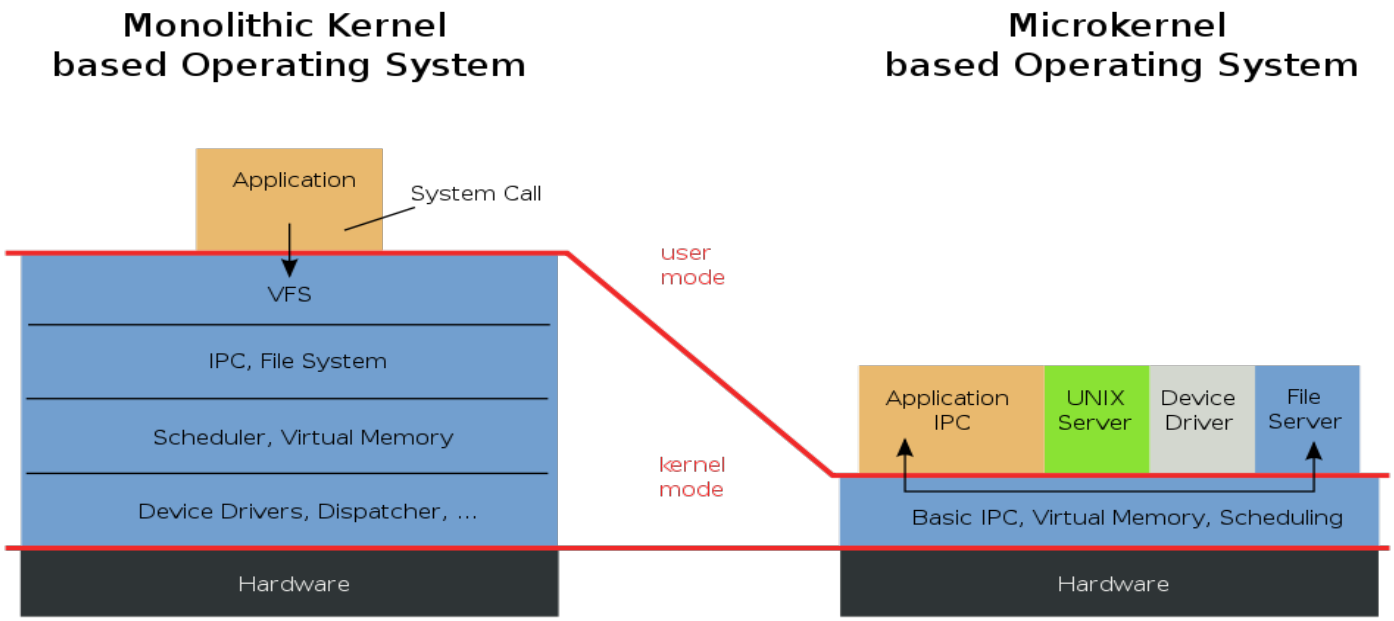
Kernels monolíticos

- Todos los servicios del S.O. se ejecutan en el mismo espacio de memoria
- Eficiente
  - Acceso flexible y completo al HW de todos los servicios dentro del kernel
- Difícil de mantener
  - Gran dependencia entre componentes y módulos. Gran número de líneas de código
- Mejora: módulos cargables

Módulos del kernel (Linux)

- Partes o componentes del kernel
- Se ejecutan en modo privilegiado
- Proporcionan funcionalidades adicionales:
  - Driver de dispositivo
  - Formato de sistema de fichero
  - Protocolo de red
  - …
- Los módulos se pueden cargar/descargar dinámicamente
  - Se integran en el espacio del kernel

Otros tipos de kernels



- Kernels híbridos
  - Organización microkernel, pero monolítica, en el mismo espacio de memoria
  - Combina ventajas e inconvenientes de ambas soluciones